



第二十章 软件架构坏味道

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第20章 软件架构坏味道

- 20.1 引言
- 20.2 典型的代码坏味道
- 20.3 典型的设计坏味道
- 20.4 本章小结

20.1 引言（I）

坏味道可能存在于程序的代码中，也可能存在于软件设计中，还可能存在于系统的模型、文档和数据中。

本章中主要讨论**代码坏味道**（code smell）
和**架构坏味道**（architectural smell）

20.1 引言 (2)

- 代码坏味道 (code smell)
 - 如果程序中的某一段代码是不稳定或者有一些潜在问题的，那么该段代码往往会包含一些明显的不太好的痕迹。我们称这些痕迹为“代码坏味或者代码异味”。

20.1 引言 (3)

- 架构坏味道 (architectural bad smell)
 - Lippert和Roock, 2007年提出
 - 他们认为架构坏味道定义与代码坏味道 (code bad smell) 类似的坏味道, 只是架构坏味道在系统粒度下出现的层次要高于代码坏味道。

20.1 引言 (4)

- 架构坏味道定义

- 架构坏味道是一种通常使用的、可以对系统生命周期特性产生消极影响的架构设计。
 - 它可能是由于在**不适当的环境下**应用了一个**不合适的解决方案**；
 - 或者在**错误的粒度层次下**应用了**某个设计抽象**等产生的，会对系统的可理解性、可测试性、可扩展性以及可重用性等产生负面影响。

20.2 典型的代码坏味道（I）

代码坏味道的类型很多，这里从三个角度把面向对象程序中可能出现的代码坏味道分成应用级的坏味道、类级的坏味道和方法级的坏味道三种类型。

20.2典型的代码坏味道（2）

- 20.2.1 应用级的坏味道

- (1) **重复代码 (Duplicated code)**：是指在一个以上的地方出现了重复的代码片段。重复的代码结构使程序变得冗长，需要重构。
 - 例如：一个父类有两个子类，这两个子类中存在相同的表达式。
 - 解决方案：对两个子类使用提炼方法，然后使用上移方法 (Pull Up Method) 将提炼出来的代码定义到父类中去。

20.2 典型的代码坏味道 (3)

• 20.2.1 应用级的坏味道

- (2) **人为复杂性 (Contrived complexity)** : 在进行简单设计就充分的情况下, 强制使用了过于复杂的设计模式。
- (3) **散弹式修改 (Shotgun surgery)** : 如果每遇到某种变化, 你都必须在许多不同的类中做出许多小修改。
 - 如果需要修改的代码散布四处, 你不但很难找到它们, 也很容易忘记某处重要的修改。
 - 这种情况应该使用移动方法 (Move Method) 和移动域 (Move Field) 的方式把所需要修改的代码放进同一个类中。

20.2典型的代码坏味道（4）

- 20.2.2类级的坏味道

- （1）**过大的类（Large class）**：如果想利用单个类做太多事，类中就会出现太多实例变量。
 - 可以使用extract class 将几个变量一起提炼至新类内。
 - 如果large class是个GUI类，可能需要把数据和行为移到一个独立的领域对象去。

20.2典型的代码坏味道（5）

- 20.2.2类级的坏味道

- （2）**依恋情结的类（Feature envy）**：即一个类的方法过多地使用了另一个的方法。
 - 某个类的函数对另一个类的兴趣高过对自己所处的类，通常的焦点就是数据，某个函数为了计算某个值，从另一个对象那儿调用几乎半打的取值函数。
 - 这时应该运用 Move Method 把它移到自己该去的地方。

20.2典型的代码坏味道（6）

- 20.2.2类级的坏味道

- (3) **过度亲密类 (Inappropriate intimacy)** :
一个类存在与另一个类的实现细节依赖。
 - 例如：两个类过于亲密，花费太多时间去探究彼此的private成分。
 - 可以采用move method和move field划清界限。
 - 可以通过把双向关联变成单项关联。

20.2典型的代码坏味道（7）

- 20.2.2类级的坏味道

- （4） **拒绝的馈赠（Refused request）**：子类仅仅使用父类中的部分方法和属性，其他来自父类的馈赠成为了累赘。
 - 问题的原因是有些人仅仅是想重用超类中的部分代码而创建了子类。但实际上超类和子类完全不同。
 - 解决方法：如果继承没有意义并且子类和父类之间确实没有共同点，可以消除继承。

20.2典型的代码坏味道（8）

- 20.2.2类级的坏味道

- (5) **冗赘类 (Lazy class)** : 一个做的事情太少的类。
 - 冗赘类出现的主要原因:
 - 1) 某个类原本对得起自己的身价, 但重构使它身形缩水, 不再做那么多工作;
 - 2) 或开发者事前规划了某些变化, 并添加一个类来应付这些变化, 但变化实际上没有发生。

20.2典型的代码坏味道（9）

- 20.2.2类级的坏味道

- （6） **数据泥团（Data clump）**： 当一群变量一起在程序的不同部分传播时会发生数据泥团现象。即：常常可以在很多地方看到相同的若干项数据。
 - 这些总是绑在一起出现的数据应该拥有属于它们自己的对象。首先找出这些数据以字段形式出现的地方，运用Extract Class 将它们提炼到一个独立对象中。

20.2典型的代码坏味道（10）

- 20.2.2类级的坏味道

- (7) **不完美的程序库类（Incomplete Library Class）**：尽管许多编程技术都建立在library classes的基础上，但是Library classes构筑者没有未卜先知的能力，不可能考虑所有可能的情况。
 - 如果只想修改library classes内的一两个方法，可以通过引入外援方法（Introduce Foreign Method）来实现；
 - 如果想要添加一大堆额外行为，就得通过引入局部扩展机制（Introduce Local Extension）来实现。

20.2典型的代码坏味道（II）

• 20.2.2类级的坏味道

- **(8) 纯数据类 (Data Class)**：所谓Data Class是指：它们拥有一些字段，以及用于访问这些字段的方法，除此之外一无长物。这样的classes只是一种“不会说话的数据容器”，它们几乎一定被其它classes过份细琐地操控着。
 - 如果这些classes内含容器类的字段，应检查它们是不是得到了恰当的封装；如果没有，就运用Encapsulate Collection把它们封装起来。
 - 对于那些不该被其他classes修改的字段，移走设置方法（Remove Setting Method）。

20.2典型的代码坏味道（12）

- 20.2.3方法级的坏味道

- 过长方法（Long method）：一个方法（或者函数、过程）含有太多行代码。
- 参数太多（Too many parameters）
- 超长标识符（Excessively long identifiers）
- 超短标识符（Excessively short identifiers）
- 数据过量返回（Excessive return of data）
- 超长代码行（Excessively long line of code）
- 等等

20.2典型的代码坏味道（13）

例如，下面就是一个超长代码行的例子：

```
new XYZ(s).doSomething(buildParam1(x),  
buildParam2(x), buildParam3(x), a +  
Math.sin(x)*Math.tan(x*y +  
z)).doAnythingElse().build().sendRequest();
```

20.3 典型的设计坏味道（I）

- 软件设计过程中，由于各种原因（人员素质、技术方法、技术债等原因）也会引起坏味道，我们称之为设计坏味道（design bad smell）。
- 架构的坏味道是一种设计级的坏味道，但是它的抽象层次更高，潜在危害更大。

20.3 典型的设计坏味道 (2)

- 根据实践经验，Garcia等定性地描述了架构坏味道，提出了11种架构坏味道：
 - (1) 连接件嫉妒 (Connector Envy) ；
 - (2) 过度分散的功能 (Scattered Functionality) ；
 - (3) 模糊接口 (Ambiguous Interface) ；
 - (4) 无关的相邻连接件 (Extraneous Adjacent Connector) ；
 - (5) 砖关注过载 (Brick Concern Overload) ；
 - (6) 砖使用过载 (Brick Use Overload) ；
 - (7) 砖循环依赖 (Brick Dependency Cycle) ；
 - (8) 未使用接口 (Unused Interface) ；
 - (9) 重复的组件功能 (Duplicate Component Functionality) ；
 - (10) 组件嫉妒 (Component Envy) ；
 - (11) 连接件链 (Connector Chain) 。

20.3.1 架构坏味道（I）

连接件嫉妒

- 如果想要制作一个MapDisplay组件，可以通过机器所在的环境利用该组件绘制出一个完整的地图。
- 而且我们希望这个组件可以接收以笛卡尔坐标表示的位置信息，并且可以自动转换成只是用正数（x, y）表示的系统屏幕坐标。

20.3.1 架构坏味道 (2)

连接件嫉妒

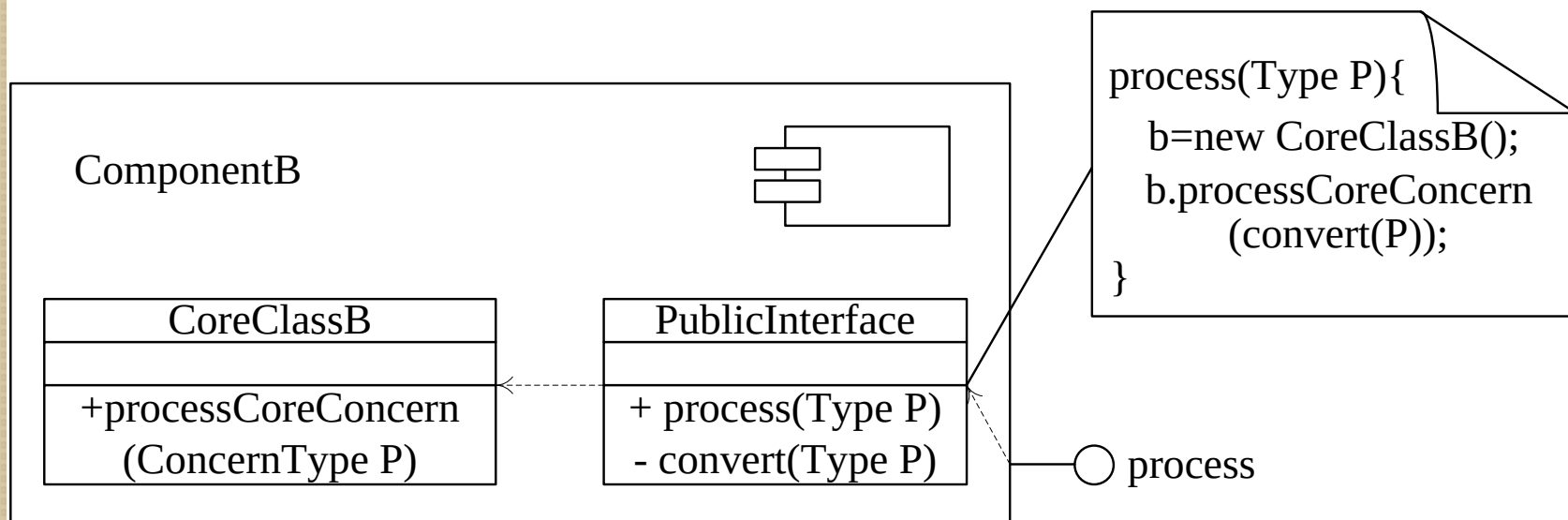


图20.2 连接件嫉妒案例2

20.3.1 架构坏味道 (3)

连接件嫉妒

- 不换机器环境的情况下，可能系统运行无差错，组件功能能够明确完成。
- 当把该组件使用在一个新的机器上时，假如该机器是使用屏幕坐标作为数据传递给 MapDisplay 组件，作为它的输入，那么组件中原来用于实现转化功能的服务就显得多余了。

20.3.1 架构坏味道（4）

连接件嫉妒

- 如果该功能仍被使用的话，可能会导致组件显示出错误的地址信息
- 而且由于组件中的适配器是封装的，使得组件单独测试该转化服务是否被调用十分困难。
- 广泛地将连接件的功能与组件的功能相连接导致的连接件嫉妒会降低组件的可重用性、可测试性和可理解性。但性能较好。

20.3.1 架构坏味道（5）

连接件嫉妒

- 如果该系统对于性能的要求远大于可维护性，那么系统中出现连接件嫉妒是可以容忍的。
- 虽然对于某些系统来说，出于某些考虑可能做出上面的权衡来保留连接件嫉妒现象，但是这种不考虑该味道对系统生命周期可维护性，可能会产生毁灭性的影响。

20.4 本章小结

- 代码的坏味道是用来帮助开发者在开发的过程中识别出具体什么地方需要进行重构的工具
- 架构的坏味道是用来告诉开发者什么时间、是什么地方对架构进行适当的重构工作。
- 架构坏味道表现出违背了传统软件工程原则的特性，比如隔离变化、分离关注点等，但是通过给出特定的可重复的形式从而使它们有能够被自动识别出的潜在可能。
- 根据架构坏味道的特性，架构师可以通过逐渐地改善小的、局部的系统区块来最终对一个大型的、复杂的系统进行改善。